PRODUCT REQUIREMENTS DOCUMENT

Facebook Pages Automation System

Automated multi-page scheduling, publishing, and first-comment workflow

Version	1.0
Date	April 2026
Author	Internal
Status	Ready for Development
Scope	Local App + n8n Workflows + Meta Graph API

This document is divided into three independent parts:

- Part 1 — Local Interface: the locally hosted web app that serves as the control panel
- Part 2 — n8n Workflows: the automation layer that handles scheduling, posting, and comments
- Part 3 — Meta Graph API Setup: the configuration of the Meta developer infrastructure

System Overview

The Facebook Pages Automation System replaces a manual daily workflow of publishing 6 posts + 2 stories across 20 Facebook Pages, each followed by a first comment containing a partner link. The system consists of three integrated layers:

Goal	Eliminate 2+ hours of manual daily work while maintaining full control over scheduling, content, and timing.
Scope	20 Facebook Pages. 6 feed posts + 2 stories per page per day. Each post followed by a delayed first comment with a link.
Post Window	15:00 to 20:00 local time. One post per hour, published simultaneously across all 20 pages.
Content Types	Photos (images with caption), Reels (short-form video), Stories (photo or video).
Revenue Logic	First comment contains a link to partner site where readers finish the story — this drives affiliate/partner traffic.
Infrastructure	Local PC runs a lightweight web app. n8n (cloud or self-hosted) runs the automation. Meta Graph API handles publishing.

NOTE — Groups vs. Pages

This system targets Facebook Pages only. The Facebook Groups API was deprecated by Meta in April 2024 and cannot be used for automation. All 20 of your accounts must be Facebook Pages, not Groups.

PART ONE

Local Interface

The control panel application running on your PC

1.1 Purpose and Technology Stack

The local app is a lightweight single-page web interface that runs on your PC via a Node.js server. You open it in a browser at localhost. It collects all the information needed for each batch of posts and sends it to your n8n instance via an HTTP webhook. It also serves as a media staging server — temporarily hosting uploaded files at a public URL that Meta's API can access during upload.

Recommended Technology Stack

Runtime	Node.js 20+
Server	Express.js (minimal, ~50 lines to configure)
Frontend	Single HTML file with vanilla JavaScript — no build step needed
Database	SQLite via better-sqlite3 (stores queue history, page list, tokens)
Media Staging	Express static file server exposing /media at your public IP or via ngrok tunnel
IDE	Google Project IDX (cloud IDE that connects to your local machine) or VS Code
Packaging	Optional: pkg or nexe to bundle into a single executable for easy launch

Why not a React app?
A single HTML file with vanilla JS is faster to set up, requires no build step, runs instantly, and is trivial to maintain. For a single-user internal tool of this scope, it is the right choice. If you later want to extend it significantly, migrating to React is straightforward.

1.2 Application Screens

Screen 1 — Daily Post Queue Builder

This is the main screen used every day. It is a structured form that defines the full day's batch. On submission, it sends a single payload to n8n.

Fields — Post Slots (repeat 6 times)

- Post Slot Label: "Post 1" through "Post 6" with the auto-calculated publish time shown (e.g. 15:00, 16:00, ...).
- Content Type: dropdown — Photo, Reel, Story.
- Media File: file upload input. Accepted formats: JPEG, PNG, MP4, MOV. Max size: 500 MB.
- Caption: multi-line text area with character counter. Max 63,206 characters (Meta limit).
- First Comment: single-line text input. This is where the partner link goes.
- Comment Delay: number input in seconds. Default: 60. Range: 30 to 3600.
- Page Selector: a multi-checkbox list of all 20 pages pulled from local SQLite. Default is all 20 selected.

Fields — Global Settings (top of form)

- Posting Date: date picker. Defaults to today.
- First Post Time: time picker. Defaults to 15:00. The app auto-calculates the remaining 5 post times at 1-hour intervals.
- Story Slot: checkbox to mark which of the 6 slots (if any) should also include stories.

Behaviour on Submit

1. Client-side validates all 6 slots are filled.
2. Uploads all media files to the local /media staging endpoint and receives back public URLs.
3. Assembles a JSON payload (schema defined in Part 2) and POSTs it to the n8n webhook URL stored in settings.
4. Displays a confirmation screen showing the queue status returned by n8n.

Screen 2 — Queue Monitor

A real-time status board showing all scheduled and in-progress posts for the current day.

- One row per post per page. Columns: Page Name, Post Slot, Scheduled Time, Status, Post ID, Comment Status.
- Status values: Scheduled, Uploading, Published, Comment Posted, Error.
- Status is polled from the local SQLite database every 10 seconds (n8n writes back to it via a second webhook).
- Error rows shown in red with the raw error message from the Meta API.
- Manual "Retry" button per row for error recovery.

Screen 3 — Page Manager

A configuration screen used once during setup and occasionally when pages are added or removed.

- Lists all pages stored in the local database with their Facebook Page ID and access token status.
- "Sync from Meta" button: calls a dedicated n8n webhook that fetches the current list of pages from the Meta Graph API using the System User token, then writes them to SQLite.
- Token status indicator: green if the stored Page Access Token is valid, red if expired or missing.
- "Refresh All Tokens" button: triggers the n8n token refresh workflow.

Screen 4 — Settings

- n8n Webhook URL: text input. This is where the app sends the daily payload.
- n8n Status Callback URL: the URL n8n calls to write status back to the local app.
- Public Media Base URL: the base URL of your media staging server (e.g. https://your-ngrok.io/media or http://your-public-ip:3000/media).
- Default Comment Delay (seconds): global default for all first comment delays.
- Default First Post Time: global default for the start of the daily window.

1.3 Media Staging Server

The Meta Graph API requires all media files to be accessible via a direct, public, unauthenticated HTTP URL at upload time. This means your files cannot be behind a login wall, cannot be Google Drive links, and cannot have redirects.

Implementation

- Express serves a /media route that maps to a local folder (e.g. /home/user/facebook-app/media/).
- When a file is uploaded through the web UI, it is saved to this folder with a UUID filename.
- The app constructs a public URL: https://your-public-url/media/<uuid>.mp4 and returns it to the frontend.
- n8n uses this URL in the Meta API upload call. Once Meta has ingested the file (usually seconds), the local copy can be deleted.
- Automatic cleanup: a daily cron job inside the app deletes media files older than 24 hours.

Making Localhost Public

You have two options for making your local media server accessible to Meta's servers:

- Option A — Static public IP: if your router allows port forwarding, forward port 3000 to your PC and use your public IP. Free, but requires router configuration.
- Option B — ngrok tunnel: run ngrok http 3000 to get a temporary public HTTPS URL. The free tier is sufficient. Update the Public Media Base URL in Settings each session (or use a paid ngrok plan with a fixed domain).

1.4 Local SQLite Database Schema

Table: pages

```
CREATE TABLE pages (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  page_id     TEXT NOT NULL UNIQUE,
  page_name   TEXT NOT NULL,
  access_token TEXT NOT NULL,
  token_expires_at TEXT,
  active      INTEGER DEFAULT 1
);
```

Table: post_queue

```
CREATE TABLE post_queue (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  batch_id        TEXT NOT NULL,
  page_id         TEXT NOT NULL,
  slot_number     INTEGER NOT NULL,
  content_type    TEXT NOT NULL,
  media_url       TEXT,
  caption         TEXT,
  first_comment   TEXT,
  comment_delay   INTEGER DEFAULT 60,
  scheduled_at    TEXT NOT NULL,
  status          TEXT DEFAULT 'pending',
  fb_post_id      TEXT,
```

```
  comment_status  TEXT DEFAULT 'pending',
  error_message   TEXT,
  created_at      TEXT DEFAULT (datetime('now'))
);
```

1.5 Webhook Payload Schema

This is the JSON structure the local app sends to the n8n webhook on form submission. n8n reads this to build its execution plan.

```
{
  "batch_id": "2026-04-25-001",
  "posting_date": "2026-04-25",
  "posts": [
    {
      "slot": 1,
      "scheduled_time": "2026-04-25T15:00:00+02:00",
      "content_type": "photo",
      "media_url": "https://your-host/media/uuid.jpg",
      "caption": "Story headline here...",
      "first_comment": "https://partner-site.com/full-story",
      "comment_delay_seconds": 60,
      "pages": [
        { "page_id": "123456789", "access_token": "EAA..." },
        { "page_id": "987654321", "access_token": "EAA..." }
      ]
    },
    { "slot": 2, "...": "..." },
    { "slot": 3, "...": "..." },
    { "slot": 4, "...": "..." },
    { "slot": 5, "...": "..." },
    { "slot": 6, "...": "..." }
  ],
  "stories": [
    {
      "scheduled_time": "2026-04-25T09:00:00+02:00",
      "content_type": "photo_story",
```

```
      "media_url": "https://your-host/media/story.jpg",
      "pages": [ "..." ]
    }
  ]
}
```

PART TWO

n8n Workflows

The automation layer that orchestrates scheduling, publishing, and commenting

2.1 n8n Setup Requirements

n8n Version	1.x (latest stable — self-hosted or n8n Cloud)
Hosting	n8n Cloud (recommended for reliability) or self-hosted on a VPS or your PC
Credentials Required	Meta Graph API credential (HTTP Header Auth with Bearer token), SQLite connection (if using n8n DB node), HTTP Request nodes for all Meta calls
Execution Mode	Main process (not queue) is fine for this workload — no concurrency issues at 20 pages
Timezone	Set n8n instance timezone to match your local timezone before building any schedule triggers

Self-hosted vs. Cloud
If n8n is running on your PC, it must be on when posts are due. n8n Cloud ($20/month) runs 24/7 without your PC. For a production publishing workflow, Cloud is recommended. If you choose self-hosted on PC, ensure the machine does not sleep during the 15:00–21:00 window.

2.2 Workflow Architecture

The system uses four distinct n8n workflows. They are designed to be independent so that a failure in one does not cascade.

Workflow 1	Ingest — receives the daily payload from the local app and writes it to the execution queue.
Workflow 2	Publisher — fires on a schedule, reads pending posts for the current hour, and publishes them to all pages.
Workflow 3	Commenter — fires after each post is confirmed published and posts the first comment after the configured delay.

Workflow 4	Token Manager — runs weekly to check and refresh Page Access Tokens before they expire.

2.3 Workflow 1 — Ingest

Purpose

Receives the JSON payload from the local app, validates it, writes each post-page combination as a row in the n8n internal data store (or your SQLite DB via HTTP), and returns a confirmation.

Trigger

- Node type: Webhook (POST)
- URL: https://your-n8n/webhook/fb-ingest
- Authentication: Header Auth with a shared secret. The local app sends this in the X-Webhook-Secret header.

Nodes in Order

5. Webhook — receives POST body from local app.
6. Validate Input — Code node. Checks: all 6 slots present, each slot has media_url, caption, pages array. Returns error 400 if invalid.
7. Split Posts — SplitInBatches node. Splits the posts array into individual post objects.
8. Split Pages — second SplitInBatches node inside the loop. For each post, splits the pages array.
9. Write to Queue — HTTP Request node POSTing to local app's /api/queue endpoint to write each row into SQLite. Alternatively, use n8n's built-in Static Data or a Google Sheet as the queue store.
10. Respond to Webhook — Respond to Webhook node. Returns 200 with queue summary: { "queued": 120, "batch_id": "..." }.

2.4 Workflow 2 — Publisher

Purpose

Runs hourly from 15:00 to 20:00. Reads all posts scheduled for the current hour and publishes them to their respective pages. Handles both photos and reels with different upload flows. Updates queue status after each publish.

Trigger

- Node type: Schedule Trigger
- Settings: Every hour at minute 00, active between 14:50 and 20:10 (to safely catch the 15:00–20:00 window).
- Alternatively: use 6 separate Schedule Trigger nodes at exactly 15:00, 16:00, 17:00, 18:00, 19:00, 20:00.

Nodes in Order

11. Schedule Trigger — fires on the hour.
12. Calculate Current Slot — Code node. Uses current timestamp to determine which slot number to publish (15:00 = slot 1, 16:00 = slot 2, etc.).
13. Fetch Pending Posts — HTTP Request to local app's /api/queue?slot={n}&status=pending, returns all page-post rows for this slot.
14. Check Empty — IF node. If no rows returned, workflow ends cleanly.
15. Split by Page — SplitInBatches node. Process one page at a time.
16. Route by Content Type — Switch node. Three branches: photo, reel, story.

Branch A — Photo

a. Upload Photo — HTTP Request POST to https://graph.facebook.com/v25.0/{page_id}/photos with params: url (media_url), caption, published: false, scheduled_publish_time (Unix timestamp of this slot time). Returns: { "id": "post_id" }.

b. Update Queue — HTTP Request to local app marking this row status = "published" with the returned post_id.

c. Trigger Commenter Workflow — HTTP Request to n8n webhook /fb-comment with: post_id, page_id, access_token, comment_text, delay_seconds.

Branch B — Reel (Video)

d. Initialize Upload Session — HTTP Request POST to https://graph.facebook.com/v25.0/{page_id}/videos with: upload_phase = "start", file_size (bytes of video file). Returns: { "upload_session_id": "...", "video_id": "..." }.

e. Upload Video Binary — HTTP Request POST to https://rupload.facebook.com/video-upload/v25.0/{upload_session_id}. Headers: Authorization: OAuth {access_token}, offset: 0, file_size: {size}. Body: binary file content fetched from media_url.

17. Wait for Processing — Wait node (30 seconds). Meta needs time to process the video.

    a. Check Upload Status — HTTP Request GET to https://graph.facebook.com/v25.0/{video_id}?fields=status. IF status.processing_progress = 100, proceed. Otherwise loop back to Wait.

    b. Publish Reel — HTTP Request POST to https://graph.facebook.com/v25.0/{page_id}/videos with: video_id, description (caption), published: false, scheduled_publish_time. Returns the post ID.

    c. Update Queue + Trigger Commenter — same as Photo branch.

Branch C — Story

d. Post Photo Story — HTTP Request POST to https://graph.facebook.com/v25.0/{page_id}/photo_stories with: url (media_url). For video stories, use /video_stories. Stories cannot be scheduled ahead of time — they publish immediately.

e. Update Queue — mark status = "published".

Error Handling

- Wrap each publish branch in a Try/Catch (Error Trigger node on a branch).
- On error: HTTP Request to local app to update queue row with status = "error" and error_message = Meta API error body.
- Error rows appear in the Queue Monitor for manual retry.

2.5 Workflow 3 — Commenter

Purpose

Receives a trigger from the Publisher workflow with post details, waits for the configured delay, then posts the first comment to the published post. Using a separate workflow prevents the Publisher from being blocked by the wait time.

Trigger

- Node type: Webhook (POST)
- URL: https://your-n8n/webhook/fb-comment
- Authentication: same shared secret as Ingest webhook.

Incoming Payload

```
{
  "post_id": "123456789_987654321",
  "page_id": "123456789",
  "access_token": "EAA...",
  "comment_text": "https://partner-site.com/full-story",
  "delay_seconds": 60
}
```

Nodes in Order

18. Webhook — receives trigger from Publisher.
19. Respond to Webhook — immediately return 200 so Publisher is not blocked. The rest of the workflow continues asynchronously.
20. Wait Node — configured to wait delay_seconds seconds. n8n resumes the execution automatically.
21. Post Comment — HTTP Request POST to https://graph.facebook.com/v25.0/{post_id}/comments with body: { "message": "{comment_text}", "access_token": "{access_token}" }. Returns: { "id": "comment_id" }.
22. Update Queue — HTTP Request to local app to mark comment_status = "posted".
23. Error Catch — on failure, update comment_status = "error" and log the message.

> **IMPORTANT — Webhook Execution Mode**
> n8n webhooks by default complete synchronously. To make the Commenter work asynchronously (so the Wait node does not block the Publisher), the Commenter webhook must use "Respond Immediately" mode in n8n webhook settings. This is available in n8n 1.x under the Webhook node settings as "When to respond" = "Immediately".

2.6 Workflow 4 — Token Manager

Purpose

Page Access Tokens derived from personal User Tokens expire in 60 days. This workflow uses a never-expiring System User Token (see Part 3) to periodically regenerate Page Access Tokens for all 20 pages and write them back to the local SQLite database.

Trigger

- Node type: Schedule Trigger
- Settings: Every Monday at 09:00.

Nodes in Order

24. Schedule Trigger.
25. Get System User Token — read the long-lived System User token from n8n Credentials store.
26. Fetch Page List — HTTP Request GET to https://graph.facebook.com/v25.0/me/accounts?access_token={system_user_token}. Returns array of pages with their current Page Access Tokens.
27. Split Pages — SplitInBatches over the returned pages array.
28. Update Token in DB — HTTP Request POST to local app's /api/pages/token endpoint with: { "page_id": "...", "access_token": "..." }.

29. Log Summary — Code node that assembles a count of updated tokens and writes it to n8n's execution log.

2.7 n8n Credentials Configuration

Store all sensitive values in n8n's built-in Credentials store, not in workflow nodes directly.

Meta System User Token	Create a "Header Auth" credential. Name: "Meta API". Value: Bearer {your-system-user-token}. Used in all Meta API HTTP Request nodes.
Webhook Shared Secret	Create a "Generic Credential" with a randomly generated 32-character string. Both the local app and n8n share this value. Sent as X-Webhook-Secret header.
Local App API Key	Create a second "Header Auth" credential for n8n calling back to the local app's /api/* endpoints.

2.8 Rate Limiting & Throttling in n8n

Even though Meta's rate limits are generous, add a small artificial delay between page iterations to avoid bursting all 20 API calls simultaneously.

- Add a Wait node set to 2 seconds between each page in the SplitInBatches loop.
- This spreads 20 page-publishes over ~40 seconds per slot, which is comfortable and avoids triggering anomaly detection.
- For video (Reel) uploads, wait 30 seconds after upload before checking processing status, and retry up to 5 times with 15-second intervals before marking as error.

PART THREE

Meta Graph API Setup

Step-by-step configuration of the Meta developer infrastructure

3.1 Prerequisites

Before starting, ensure you have the following in place:

- A personal Facebook account that is an admin of all 20 Pages.
- A Meta Business Manager account at business.facebook.com. All 20 Pages must be added to this Business Manager.
- A valid phone number and business email for Meta verification.
- A publicly accessible URL for your privacy policy (required for App Review). A single-page Notion doc or GitHub Pages site is sufficient.

3.2 Step 1 — Create a Meta Developer App

30. Go to developers.facebook.com and sign in with your personal Facebook account.
31. Click "My Apps" in the top right, then "Create App".
32. Select "Business" as the app type. This gives you access to the Pages API and the publishing permissions you need.
33. Fill in the app name (e.g. "FB Pages Scheduler"), your contact email, and link it to your Business Manager account.
34. After creation, you will land on the App Dashboard. Note your App ID and App Secret — you will need these in Step 3.

App Type Matters

Select "Business" type. Do not select "Consumer" (limited permissions) or "None" (developer tools only). The Business type grants access to pages_manage_posts and pages_manage_engagement which are required for this workflow.

3.3 Step 2 — Add Required Products to Your App

In the App Dashboard left sidebar, click "Add Product" and add the following:

- Facebook Login — required to generate User Access Tokens during initial setup.
- Pages API — the core product that grants publishing capabilities.
- Webhooks — needed if you want real-time publish confirmation for the comment trigger (optional but recommended).

3.4 Step 3 — Create a System User

A System User is a non-human administrator account in Business Manager that generates never-expiring access tokens. This is the correct credential for automated workflows — never use your personal token in production.

35. Go to business.facebook.com.
36. Navigate to Business Settings > Users > System Users.
37. Click "Add" and create a System User. Name it something like "fb-pages-bot". Role: Admin.
38. After creation, click the System User and click "Add Assets". Add all 20 Pages and grant "Full Control" on each.
39. Click "Generate New Token" on the System User. Select your App. Check the following permissions:
    - pages_show_list
    - pages_manage_posts
    - pages_manage_engagement
    - pages_read_engagement
    - pages_read_user_content
    - business_management
40. Copy and store the generated token securely. This is your System User Token. It does not expire.

Token Security
Store the System User Token only in n8n's Credentials store and your local SQLite database's settings table. Never commit it to a git repository. Never expose it in client-side code. Treat it like a database password.

3.5 Step 4 — Get Page Access Tokens for All 20 Pages

Page Access Tokens are page-specific tokens derived from your System User Token. Each API call to publish to a page must use that page's own token.

41. Make a GET request to the following URL (replace {system_user_token} with your token):

```
GET
https://graph.facebook.com/v25.0/me/accounts?access_token={system_user_token}
```

42. The response will be a JSON array. Each object contains:

```
{
  "name": "Your Page Name",
  "access_token": "EAA...",   <- This is the Page Access Token
  "category": "Media/News",
  "id": "123456789"           <- This is the Page ID
}
```

43. Store the id and access_token for each page in your local SQLite pages table.
44. These tokens, when derived from a System User token, do not expire. The Token Manager workflow (Workflow 4) refreshes them weekly as a safety measure.

3.6 Step 5 — Request App Permissions (App Review)

To use the publishing permissions in production (beyond your own test account), your app must pass Meta's App Review. This applies because pages_manage_posts is a restricted permission.

What App Review Requires

- Detailed use case explanation for each permission requested.
- A screencast video (under 3 minutes) showing a user logging in and your app publishing a post to a page.
- A publicly accessible privacy policy URL.
- App must be in "Live" mode (not Development mode) before submission.

Permissions to Request

Permission Scope	Required For	Review Level

`pages_show_list`	Fetch all managed pages and their IDs	Standard Review
`pages_manage_posts`	Create, schedule, and publish feed posts, reels, and stories	Standard Review
`pages_manage_engagement`	Post comments on page posts (first comment)	Standard Review
`pages_read_engagement`	Read post IDs after publish for comment targeting	Standard Review
`pages_read_user_content`	Verify post status after scheduling	Standard Review
`business_management`	Access Business Manager assets (for System User setup)	Advanced Review

App Review Tips

- In your use case description, be explicit: "This is an internal tool. No external users will authenticate. I am the sole admin of all 20 Pages managed by this app."
- In the screencast: show the full flow — login, select a page, upload a photo, schedule a post, then post a comment on it.
- Internal-use apps with no external users typically have faster reviews (1–3 weeks vs. 4–6 weeks for public apps).
- If rejected, read the rejection note carefully — it will specify exactly what was missing. Resubmit promptly.

3.7 Step 6 — Configure Webhooks (Optional but Recommended)

Meta Webhooks send a real-time POST notification to your server when a scheduled post is published. Using this instead of a fixed Wait node in n8n is more reliable for timing the first comment accurately.

45. In the App Dashboard, click "Webhooks" in the sidebar.
46. Click "Add New Subscription" and select "Page" as the object.
47. Enter your n8n webhook URL for the callback URL field (e.g. https://your-n8n/webhook/fb-publish-event).
48. Enter a verify token (any random string — must match what n8n expects).
49. Subscribe to the feed field — this fires when a new post appears on the page's feed (including scheduled posts going live).
50. In n8n, build a Webhook trigger that: receives the event, extracts the post_id and page_id, looks up the matching row in the queue, and triggers the Commenter workflow with the correct delay.

3.8 Core API Endpoints Reference

Publish a Photo Post

Endpoint	Method	Key Parameters
`/v25.0/{page_id}/photos`	POST	url (media URL), caption, published (bool), scheduled_publish_time (Unix ts), access_token
`/v25.0/{page_id}/feed`	POST	message, link (optional), published, scheduled_publish_time, access_token

Publish a Reel (Video)

Endpoint	Method	Key Parameters
`/v25.0/{page_id}/videos`	POST (init)	upload_phase: "start", file_size, access_token
`/v25.0/{session_id}`	POST (upload)	Host: rupload.facebook.com — binary body + headers: offset, file_size
`/v25.0/{page_id}/videos`	POST (publish)	video_id, description, published, scheduled_publish_time, access_token
`/v25.0/{video_id}`	GET (status)	fields: status — poll until status.processing_progress = 100

Publish a Story

Endpoint	Method	Key Parameters
`/v25.0/{page_id}/photo_stories`	POST	url (image URL), access_token — publishes immediately, no scheduling
`/v25.0/{page_id}/video_stories`	POST	video_id (from upload session), access_token — publishes immediately

Post a Comment

Endpoint	Method	Key Parameters
`/v25.0/{post_id}/comments`	POST	message (the comment text / link), access_token

Get Page List

Endpoint	Method	Key Parameters

`/v25.0/me/accounts`	GET	access_token (System User token) — returns all pages with their tokens and IDs

3.9 API Version Management

Meta deprecates old Graph API versions roughly every 2 years. The current version as of April 2026 is v25.0. You should:

- Hardcode the version string (v25.0) in a single config variable in n8n, not in every node individually. Updating one variable upgrades all calls.
- Subscribe to Meta's developer changelog at developers.facebook.com/docs/graph-api/changelog to receive deprecation notices.
- When a new major version is released, test your workflows in n8n's test execution mode against the new version before switching.
- Meta gives at minimum 2 years of support per version, so you have ample migration time.

3.10 Testing Before Going Live

Test with a Single Page First

51. Set up the full stack for one page only.
52. Post a single photo to that one page and verify: the post appears, the comment appears after the delay, the queue status updates correctly.
53. Then test a Reel. Video processing can take 1–5 minutes — confirm your polling loop handles it correctly.
54. Once single-page works reliably, expand to all 20.

Meta's Graph API Explorer

Use the Graph API Explorer at developers.facebook.com/tools/explorer to test individual API calls interactively before building them into n8n. You can select your app, generate a test token, and make live API calls from the browser.

Development Mode vs. Live Mode

- While your app is in Development mode, only you (the app admin) can use it. This is fine for the entire setup phase.
- Switch to Live mode before submitting for App Review and before using the app for production posting.

- In Live mode without approved permissions, the API will return permission errors. Wait for App Review approval before production use.

3.11 Common Errors and Resolutions

Error 100 — Invalid Parameter	Usually means a missing required field. Check that scheduled_publish_time is a Unix timestamp (not ISO date), media URL is publicly accessible, and access_token is for the correct page.
Error 200 — Permissions Error	The Page Access Token does not have the required permission. Run the Token Manager workflow to refresh tokens. Check the System User has "Full Control" on the page in Business Manager.
Error 32 — Rate Limit Reached	Rare at your scale (see rate limit math in Overview). If it occurs, add longer delays between page iterations in n8n. Check the X-Page-Usage response header to monitor usage.
Error 190 — Token Expired	The access token has expired. For personal User Tokens this happens in 60 days. This should not occur if using System User tokens. Run Token Manager workflow immediately.
Video stuck in processing	Poll /v25.0/{video_id}?fields=status. If status is ERROR, the video file is likely in an unsupported format or codec. Re-encode to H.264 MP4 and retry. If IN_PROGRESS after 10 minutes, the upload may have been partial — restart the upload session.
Media URL rejected	Meta rejected the image/video URL. Ensure it is a direct link (no redirects), HTTPS, publicly accessible, and the server returns the correct Content-Type header (image/jpeg, video/mp4, etc.).

Appendix — Implementation Checklist

Part 1 — Local App

55. Initialize Node.js project: npm init, install express, better-sqlite3, multer, uuid, node-cron.
56. Create SQLite database and run CREATE TABLE scripts for pages and post_queue.
57. Build Express server with routes: GET / (serve HTML), POST /api/upload (media staging), POST /api/queue (write from n8n), GET /api/queue (read for queue monitor), POST /api/pages/token (token update from n8n).
58. Build the HTML frontend: Queue Builder form, Queue Monitor table (auto-refresh), Page Manager, Settings.
59. Configure media staging: serve /media static folder, implement 24-hour cleanup cron.
60. Set up ngrok or port forwarding and test that Meta can reach a file at your public URL.
61. Test the full form submission: submit form, verify payload appears in n8n execution log.

Part 2 — n8n

62. Create all four workflows: Ingest, Publisher, Commenter, Token Manager.
63. Store System User Token in n8n Credentials as "Meta API" Header Auth.
64. Store Webhook Shared Secret in n8n Credentials.
65. Test Workflow 1 (Ingest): send a sample payload via curl, verify rows appear in SQLite.
66. Test Workflow 2 (Publisher — Photo branch): trigger manually, verify a test photo posts to one page.
67. Test Workflow 2 (Publisher — Reel branch): upload a short test video, verify reel appears.
68. Test Workflow 3 (Commenter): post a test comment, verify it appears as first comment after delay.
69. Test Workflow 4 (Token Manager): trigger manually, verify tokens are refreshed in SQLite.
70. Configure Schedule Triggers for hourly publishing window and weekly token refresh.

Part 3 — Meta Graph API

71. Create Meta Developer account and Business app at developers.facebook.com.
72. Add all 20 Pages to Business Manager and verify admin access.
73. Create System User in Business Manager, add all 20 pages as assets with Full Control.
74. Generate System User Token with all required permissions. Store securely in n8n.
75. Call /me/accounts to fetch Page Access Tokens for all 20 pages. Write to SQLite.

76. Test all required API calls in the Graph API Explorer before building n8n nodes.
77. Record screencast video for App Review (3 minutes max, show full publish + comment flow).
78. Write privacy policy page and host it publicly.
79. Switch app to Live mode and submit App Review with all required permissions.
80. Once approved, do a full end-to-end test with all 20 pages and all 6 post slots.
81. Configure optional Webhooks for real-time publish events.

Estimated Setup Timeline

Local App Development	2–4 days (one-time)
n8n Workflow Building	2–3 days (one-time)
Meta API Setup & Testing	1–2 days (one-time)
Meta App Review (waiting period)	1–3 weeks (depends on Meta review queue)
Full End-to-End Testing	1–2 days after App Review approval
Total Time to Production	3–6 weeks from start